

Scaled Dot-Product Attention, Worked Out by Hand

Match queries to keys, average the values — with numbers

A complete numerical walk-through of the operation at the heart of every Transformer: $\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k})V$. We start with the cleanest possible case — one query against two keys — and then run a full three-token self-attention layer, computing the score matrix, the scaling, the row-wise softmax, and the value mixture, every entry exact. Nothing is hand-waved: every number below was produced by the NumPy/PyTorch reference program in Appendix A, and cross-checked against `torch.nn.functional.scaled_dot_product_attention`.

What this document does. It treats attention as a *soft dictionary lookup*. A query asks “who is relevant to me?”; each key answers with a similarity score; softmax turns those scores into weights that sum to one; the output is the weighted average of the values. We do the arithmetic on tiny 2-dimensional vectors so every dot product, exponential, and average is checkable by hand.

How to read this. Each step is isolated with a *Mini-example* box on the cleanest numbers, alongside the running example. The single-query case (Section 2) builds all the intuition; the 3×3 self-attention (Sections 4–5) shows the same machinery applied to a whole sequence at once. Section 6 derives *why* we divide by $\sqrt{d_k}$.

Concepts covered, in order: query/key/value as a soft lookup · the dot product as a similarity score · scaling by $1/\sqrt{d_k}$ · the softmax that turns scores into a probability distribution · the value average · the full $QK^\top$ score matrix · row-wise softmax into a weight matrix · multiplying by V · why large d_k without scaling kills gradients · permutation-equivariance and why position must be injected separately.

1 The setup: attention as a soft dictionary lookup

A Python dictionary is a *hard* lookup: you hand it one key, it returns the one value stored under exactly that key. Attention is the *soft* version. You hand it a **query** q ; it compares q against *every* stored **key** k_i , gets a similarity score, turns those scores into weights w_i that sum to one, and returns a *blend* of all the **values** v_i :

$$\text{Attention}(q, K, V) = \sum_i w_i v_i, \quad w = \text{softmax}\left(\frac{qK^\top}{\sqrt{d_k}}\right).$$

Three roles, three vectors per token. The query is “what am I looking for?”; the key is “what do I offer as a match?”; the value is “what I actually contribute once matched”.

Symbol	Role	Shape in a layer
q (query)	what this position is searching for	$(1, d_k)$
K (keys)	one row per position, its match-signature	(T, d_k)
V (values)	one row per position, its payload	(T, d_v)
w (weights)	softmax of scaled scores, sums to 1	$(1, T)$
output	$\sum_i w_i v_i$, a blend of values	$(1, d_v)$

Intuition

Reading a log to flag an anomaly, you do not compress all 1000 events into one number. To understand event 850 you *look back* and focus on the few events that matter — say 10, 200, 847. Attention is exactly that focusing made differentiable: the query for event 850 scores high against the keys of the relevant events, softmax concentrates the weight there, and the output is mostly those events’ values. The dot product is the “how relevant?” meter; softmax is the “focus” knob.

2 Step 1 — One query, two keys: the cleanest case

Take a single query and two keys in $d_k = 2$ dimensions, chosen so the answer is obvious by eye: the query points along the x -axis, key 1 is *aligned* with it, key 2 is *orthogonal*.

$$q = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad k_1 = \begin{bmatrix} 2 \\ 0 \end{bmatrix} \text{ (aligned)}, \quad k_2 = \begin{bmatrix} 0 \\ 2 \end{bmatrix} \text{ (orthogonal)}, \quad v_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad v_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

The score is a dot product. Similarity between q and each key is the inner product $q \cdot k_i$ — large when they point the same way, zero when orthogonal:

$$q \cdot k_1 = 1 \cdot 2 + 0 \cdot 0 = 2, \quad q \cdot k_2 = 1 \cdot 0 + 0 \cdot 2 = 0 \quad \implies \quad \text{scores} = \begin{bmatrix} 2 & 0 \end{bmatrix}.$$

The aligned key scores 2; the orthogonal key scores 0. Exactly the ordering we want.

Scale by $1/\sqrt{d_k}$. Before softmax we divide by $\sqrt{d_k} = \sqrt{2}$ (Section 6 explains why). With $1/\sqrt{2} = 0.707$:

$$\frac{1}{\sqrt{2}} \begin{bmatrix} 2 & 0 \end{bmatrix} = \begin{bmatrix} 1.414 & 0 \end{bmatrix}.$$

Softmax turns scores into weights. The softmax of a vector s is $\text{softmax}(s)_i = e^{s_i} / \sum_j e^{s_j}$. With $s = [1.414, 0]$:

$$e^{1.414} = 4.114, \quad e^0 = 1, \quad \text{sum} = 5.114, \quad w = \left[\frac{4.114}{5.114}, \frac{1}{5.114} \right] = \boxed{[0.804, 0.196]}.$$

The aligned key takes 80.4% of the weight, the orthogonal key 19.6% — and the two sum to 1, so w is a genuine probability distribution over the keys.

Output = weighted average of values.

$$\text{output} = 0.804 v_1 + 0.196 v_2 = 0.804 \begin{bmatrix} 1 \\ 0 \end{bmatrix} + 0.196 \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \boxed{\begin{bmatrix} 0.804 \\ 0.196 \end{bmatrix}}.$$

The result leans heavily toward v_1 (the value of the matched key) but keeps a small trace of v_2 . That “mostly v_1 , a little v_2 ” is the soft lookup in action: not a hard pick, a weighted blend.

Mini-example — this operation in isolation

Softmax sharpening. Softmax does not just normalise — it *exaggerates* gaps, and how much depends on the scale of the inputs. Take $z = [2, 1, 0]$:

$$\text{softmax}(z) = [0.665, 0.245, 0.090].$$

Double the inputs (sharper) and halve them (softer):

$$\text{softmax}(2z) = [0.867, 0.117, 0.016], \quad \text{softmax}(0.5z) = [0.506, 0.307, 0.186].$$

Bigger inputs $\Rightarrow$ more peaked (closer to one-hot); smaller inputs $\Rightarrow$ flatter (closer to uniform). Scaling the scores *is* a temperature knob on the focus.

3 Step 2 — A full three-token self-attention layer

In *self*-attention every token produces its own query, key, and value, and every token attends to every token (including itself). We take $T = 3$ tokens, still $d_k = d_v = 2$, and — to keep the focus on the attention mechanism rather than the projections — set $Q = K = V$ to the same three rows:

$$Q = K = V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}.$$

Row 0 is a token pointing along x , row 1 along y , row 2 diagonally. We now run all three queries at once: the matrix form computes *every* query-against-every-key score in one matrix product.

Watch out

Self-attention is **permutation-equivariant**: shuffle the input rows and the output rows shuffle the same way, but the *values* are unchanged — nothing in $\text{softmax}(QK^\top / \sqrt{d_k})V$ knows “row 0 came before row 1”. A log is not a bag; order carries meaning. That is why a Transformer must *add a positional encoding* to the token vectors before this layer. Attention mixes content; position has to be injected separately or it is simply lost.

4 Step 3 — The score matrix $QK^\top$ and its scaling

Stacking all queries as rows of Q and all keys as rows of K , the matrix $S = QK^\top$ holds in entry S_{ij} the dot product of query i with key j :

$$S = QK^\top = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}.$$

Read row 0: query 0 = (1, 0) scores 1 with key 0 (itself), 0 with key 1 (orthogonal), 1 with key 2 (diagonal, partial overlap). The diagonal entries $S_{00} = 1$, $S_{11} = 1$, $S_{22} = 2$ are each token's self-similarity — largest for the longer diagonal vector.

Scale by $1/\sqrt{d_k} = 1/\sqrt{2} = 0.707$. Multiply every entry:

$$\frac{S}{\sqrt{2}} = \begin{bmatrix} 0.707 & 0 & 0.707 \\ 0 & 0.707 & 0.707 \\ 0.707 & 0.707 & 1.414 \end{bmatrix}.$$

Intuition

$QK^\top$ is a full $T \times T$ table of “how much does token i want to listen to token j ”. Reading it by rows tells you, for each query, where its attention will go before softmax even runs. Self-similarity sits on the diagonal; off-diagonal entries are cross-token affinities. Everything downstream is just normalising this table into probabilities and using it to average V .

5 Step 4 — Row-softmax to weights, then mix the values

Softmax is applied *independently to each row* of the scaled score matrix, so each query gets its own probability distribution over the three keys. Take row 0, $[0.707, 0, 0.707]$. Exponentiate (we may subtract the row max 0.707 for numerical stability without changing the result):

$$e^{0.707-0.707} = 1, \quad e^{0-0.707} = 0.493, \quad e^{0.707-0.707} = 1, \quad \text{sum} = 2.493,$$

$$w_0 = \left[\frac{1}{2.493}, \frac{0.493}{2.493}, \frac{1}{2.493} \right] = [0.401, 0.198, 0.401].$$

Token 0 splits its attention evenly between itself and the diagonal token 2 (both scored 0.707) and gives less to the orthogonal token 1. Doing the same to every row:

$$W = \text{softmax}_{\text{row}}\left(\frac{S}{\sqrt{2}}\right) = \begin{bmatrix} 0.401 & 0.198 & 0.401 \\ 0.198 & 0.401 & 0.401 \\ 0.248 & 0.248 & 0.503 \end{bmatrix}, \quad \text{each row sums to 1.}$$

Row 2 (the diagonal token, whose self-score 1.414 is the largest anywhere) keeps the most weight on itself, 0.503, and splits the rest equally.

Output = WV . The attention output is the weight matrix times the values — each output row is that query's weighted average of the value rows:

$$O = WV = \begin{bmatrix} 0.401 & 0.198 & 0.401 \\ 0.198 & 0.401 & 0.401 \\ 0.248 & 0.248 & 0.503 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 0.802 & 0.599 \\ 0.599 & 0.802 \\ 0.752 & 0.752 \end{bmatrix}.$$

Check row 0 by hand: first coordinate = $0.401 \cdot 1 + 0.198 \cdot 0 + 0.401 \cdot 1 = 0.802$; second = $0.401 \cdot 0 + 0.198 \cdot 1 + 0.401 \cdot 1 = 0.599$. The symmetry of the inputs shows up in the output: rows 0 and 1 are mirror images, and row 2 is perfectly balanced at (0.752, 0.752).

When this is the right choice

Use the matrix form $\text{softmax}(QK^\top/\sqrt{d_k})V$ whenever you have the whole sequence in hand — training, and inference over a fixed context. One $QK^\top$ computes *all* pairwise scores in a single batched matmul, which is why Transformers train in parallel across positions. (Streaming token-by-token generation reuses cached K, V , but the arithmetic per step is identical to one row above.)

6 Step 5 — Why divide by $\sqrt{d_k}$?

The scaling is not cosmetic. A score is a sum of d_k products, $q \cdot k = \sum_{m=1}^{d_k} q_m k_m$. If the components are roughly independent with unit variance, the *variance* of that sum grows like d_k , so its typical magnitude (the standard deviation) grows like $\sqrt{d_k}$:

$$\text{std}(q \cdot k) \approx \sqrt{d_k}.$$

d_k	2	8	64	512
typical score $\sim \sqrt{d_k}$	1.414	2.828	8.000	22.627

Left unscaled, as d_k grows the gaps between scores blow up, and — from the sharpening mini-example — softmax collapses toward one-hot. Concretely, suppose the top score equals $\sqrt{d_k}$ and the runner-up is 0:

$$d_k = 2 : \text{softmax}([1.414, 0]) = [0.804, 0.196]; \quad d_k = 8 : [0.944, 0.056]; \quad d_k = 64 : [0.99966, 0.00034].$$

By $d_k = 64$ the distribution is essentially one-hot. A near one-hot softmax has *vanishing gradient* (the off-peak derivatives are nearly zero), so learning stalls. Dividing by $\sqrt{d_k}$ exactly cancels the $\sqrt{d_k}$ growth, holding the scaled score near unit size for *any* d_k — the scaled column below is constant:

$$\frac{\sqrt{d_k}}{\sqrt{d_k}} = 1 \text{ for all } d_k \implies \text{softmax}([1, 0]) = [0.731, 0.269] \text{ regardless of } d_k.$$

Watch out

Skip the $1/\sqrt{d_k}$ and at realistic head sizes ($d_k = 64$ is standard) the softmax saturates: weights become almost one-hot, the Jacobian of softmax goes to zero, and gradients to Q and K vanish — the layer stops training. The scale factor is what keeps attention in the responsive, non-saturated regime as d_k scales up. It is a fixed constant, not a learned parameter.

7 The whole computation, at a glance

#	Stage	Formula	Result (3-token example)
1	scores	$S = QK^\top$	$[[1, 0, 1], [0, 1, 1], [1, 1, 2]]$
2	scale	$S/\sqrt{d_k}, d_k=2$	multiply every entry by 0.707
3	weights	$W = \text{softmax}_{\text{row}}(\cdot)$	rows $[0.401, 0.198, 0.401] \dots$
4	check	each row of W sums to	1
5	output	$O = WV$	$[[0.802, 0.599], [0.599, 0.802], [0.752, 0.752]]$

These five lines *are* scaled dot-product attention. Multi-head attention runs h copies of this on different learned projections of the same tokens and concatenates the outputs; the per-head arithmetic is exactly what is above.

8 Equation cheat-sheet

Attention	$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$
Score matrix	$S = QK^\top, \quad S_{ij} = q_i \cdot k_j$
Scale	divide S by $\sqrt{d_k}$ (a fixed constant)
Softmax (per row)	$w_i = e^{s_i} / \sum_j e^{s_j}, \quad \sum_i w_i = 1$
Output	$O = WV, \quad O_i = \sum_j W_{ij} v_j$
Why $\sqrt{d_k}$	$\text{std}(q \cdot k) \approx \sqrt{d_k}$; cancel it to avoid one-hot
Permutation-equiv.	shuffle inputs $\Rightarrow$ outputs shuffle; add positions separately

Appendix A Reference implementation

Every number above is reproducible from the following program. The NumPy block computes the mechanism step by step; the final lines confirm it matches PyTorch's built-in `scaled_dot_product_attention` to machine precision.

```
import numpy as np, torch
import torch.nn.functional as F

def softmax(x):
    x = np.asarray(x, dtype=np.float64)
    e = np.exp(x - x.max(axis=-1, keepdims=True))
    return e / e.sum(axis=-1, keepdims=True)

# ---- Part A: one query, two keys (aligned vs orthogonal) ----
q = np.array([1., 0.]); dk = 2
k1, k2 = np.array([2., 0.]), np.array([0., 2.])
v1, v2 = np.array([1., 0.]), np.array([0., 1.])
scores = np.array([q @ k1, q @ k2])          # [2, 0]
scaled = scores / np.sqrt(dk)                # [1.414, 0]
w = softmax(scaled)                          # [0.804, 0.196]
out = w[0]*v1 + w[1]*v2                     # [0.804, 0.196]
print("Part A weights:", w, " output:", out)

# ---- Part B: full 3-token self-attention ----
Q = K = V = np.array([[1., 0.], [0., 1.], [1., 1.]])
S = Q @ K.T                                # [[1,0,1], [0,1,1], [1,1,2]]
W = softmax(S / np.sqrt(2))                 # row-softmax of scaled scores
O = W @ V                                   # [[.802, .599], [.599, .802], [.752, .752]]
print("scores S =\n", S)
print("weights W =\n", np.round(W, 3))
print("output O =\n", np.round(O, 3))

# ---- Part C: softmax sharpening (temperature via scale) ----
z = np.array([2., 1., 0.])
print(np.round(softmax(z), 3))              # [0.665 0.245 0.090]
print(np.round(softmax(2*z), 3))            # [0.867 0.117 0.016]    sharper
print(np.round(softmax(0.5*z), 3))          # [0.506 0.307 0.186]    softer

# ---- Part D: why 1/sqrt(d_k): unscaled softmax -> one-hot ----
for d in (2, 8, 64):
    s = np.sqrt(d)                          # typical top score grows ~ sqrt(d)
    print(d, np.round(softmax([s, 0.]), 5)) # 64 -> [0.99966 0.00034]

# ---- Part E: cross-check against PyTorch's built-in ----
Qt = torch.tensor(Q); Kt = torch.tensor(K); Vt = torch.tensor(V)
O_torch = F.scaled_dot_product_attention(Qt, Kt, Vt).numpy()
assert np.allclose(O_torch, O)              # exact match
print("torch SDPA matches numpy:", np.allclose(O_torch, O))
```

— end of document —